模块 1：Grounding — 感知的策略空间

模块概述

核心问题： 如何让模型「看到」代码库？看什么、看多少、怎么看？

时长： 2 - 2.5 小时

前置知识： LLM 基础原理、Prompt Engineering 基本概念、至少使用过一种 AI 编程工具

在构建 coding agent 的工程实践中，大多数团队首先关注的是模型的推理能力——选择更强的模型、设计更好的 prompt、优化 chain-of-thought。但实证研究反复证明：**agent 的瓶颈往往不在「想」，而在「看」**。一个 GPT-4 级别的模型，如果只能看到一个函数片段，其修复 bug 的能力远不如一个 GPT-3.5 级别的模型看到完整的调用链和类型定义。

本模块将系统梳理 coding agent 的「感知层」（Grounding Layer）——从暴力检索到结构化分析，从向量检索到混合方案——帮助你建立一个完整的策略空间认知，并在工程实践中做出合理的 trade-off 选择。

1.1 为什么 Grounding 是瓶颈

SWE-agent 的核心洞察

2024 年，Princeton NLP Group 发表了 SWE-agent 论文（Yang et al., 2024），在 SWE-bench 上取得了当时的 SOTA 成绩。但这篇论文最重要的贡献不是分数本身，而是一个反直觉的发现：

> **Agent-Computer Interface (ACI) 的设计比底层模型的选择更重要。**

论文通过系统性的消融实验（ablation study）证明：

变量	修改方式	性能变化
底层模型	GPT-4 → Claude 3 Opus	变化约 ±3%
文件查看方式	滚动式 → 搜索式	**变化超过 10%**
错误反馈格式	原始 traceback → 结构化摘要	**变化超过 8%**
编辑工具设计	自由编辑 → 模板化编辑	**变化超过 12%**

换句话说，**你给模型提供什么信息，以什么格式提供，比你用哪个模型更重要。**

为什么会这样？

这个现象背后有三层原因：

第一层：信息不对称（Information Asymmetry）

一个典型的生产级代码库有几十万到几百万行代码。即使是 200K token 的上下文窗口，也只能容纳约 15 万行代码——不到大型项目的 10%。Agent 必须在极度不完整的信息中做决策。

```
代⍰⍰⍰量: ~500,000 行
上下文窗口: ~150,000 行 (200K tokens)
利用率上限: 30%
⍰⍰利用率 (扣除 prompt, history, tool output): ~10-15%
```

第二层：检索精度直接影响推理质量

LLM 的推理不是在真空中进行的——它是基于上下文中的信息做推理。如果上下文中缺少关键信息（比如一个被覆盖的方法、一个隐式的类型转换），模型的推理再强也无法弥补。这类似于人类程序员：如果你只看一个函数的实现而不知道它的调用方、不知道传入参数的实际类型，你也很难判断这个函数有没有 bug。

第三层：Token 效率是一个工程约束

每次 API 调用都有 token 成本。假设一次完整的 bug 修复任务需要 10 轮交互，每轮平均消耗 50K input tokens：

```
⍰次任⍰成本 = 10 ⍰ × 50K input tokens × $3/M tokens = $1.50
每天 100 个任⍰ = $150/天
每月 = $4,500
```

如果通过更好的 grounding 策略减少 30% 的 token 消耗，每月节省 $1,350。在规模化部署中，这个数字会急剧放大。

Grounding 的本质是一个搜索问题

从算法角度看，grounding 本质上是在一个巨大的信息空间中做搜索：

```
⍰定：
  - 代⍰⍰⍰C = {f₁, f₂, ..., f⍰}（所有文件）
  - 任⍰描述 T（自然⍰言）
  - 上下文⍰算 B（token 限制）

目⍰：
 找到子集 S ⊆ C，使得：
  1. |tokens(S)| ≤ B            （⍰足⍰算⍰束）
  2. 包含解决 T 所需的所有关⍰信息  （信息完整性）
  3. 不包含无关信息            （信息精确性）
```

这是一个 NP-hard 的优化问题——我们无法在多项式时间内找到最优解。不同的 grounding 策略，本质上是这个搜索问题的不同近似算法。

一个直观的思维模型

想象你是一个被空投到陌生城市的侦探，需要找到一个特定的人。你有三种策略：

1. **暴力搜索：** 挨家挨户敲门（Glob/Grep）——一定能找到，但极其耗时
2. **查阅户籍系统：** 通过结构化数据库定位（AST/LSP）——高效但需要系统存在且可用
3. **询问当地人：** 基于语义相似性找到可能的位置（RAG）——快速但可能不准确
4. **混合策略：** 先问当地人缩小范围，再查户籍确认（Hybrid）——通常是实践中的最优解

接下来四个章节，我们逐一展开每种策略。

1.2 策略 1：暴力检索（Brute-force Retrieval）

Claude Code 的方法论

Claude Code（Anthropic 官方的 CLI 编程助手）采用的是最直接的 grounding 策略：**给模型提供 Glob、Grep、Read 三个工具，让模型自己决定读什么。**

这种方法的哲学是：与其预先构建复杂的索引系统，不如信任模型的判断能力，让它像人类程序员一样通过搜索和阅读来理解代码库。

工具 1：Glob — 文件模式匹配

Glob 工具根据模式（pattern）在文件系统中搜索文件，返回匹配的文件路径列表。

核心参数：

参数	说明	示例
`pattern`	Glob 模式，支持 `*`，`**`，`?`，`[]`	`**/*.ts`，`src/**/test_*.py`
`path`	搜索起始目录（可选）	`/workspace/project/src`

工作原理：

```
⍰入: pattern = "src/**/*.test.ts"

匹配⍰程:
  src/                       → ⍰入
  src/utils/                 → ⍰入（匹配 **）
  src/utils/helper.ts        → 不匹配（缺少 .test）
  src/utils/helper.test.ts   → ✓ 匹配
  src/components/             → ⍰入
  src/components/Button.tsx  → 不匹配
  src/components/Button.test.ts → ✓ 匹配

⍰出:
  src/utils/helper.test.ts
  src/components/Button.test.ts
```

实际 Agent 调用示例：

假设 agent 收到任务「修复登录页面的表单验证 bug」，它的第一步通常是用 Glob 定位相关文件：

```
// Agent 的工具调用
{
  "tool": "Glob",
  "parameters": {
    "pattern": "**/*login*",
  }
}
```

返回结果可能是：

```
src/pages/LoginPage.tsx
src/pages/LoginPage.test.tsx
src/components/LoginForm.tsx
src/api/login.ts
src/styles/login.module.css
```

Agent 接着可能缩小范围：

```
{
  "tool": "Glob",
  "parameters": {
    "pattern": "**/*valid*"
  }
}
```

```
src/utils/validation.ts
src/utils/validation.test.ts
src/hooks/useFormValidation.ts
```

关键观察： Glob 本身只返回文件路径，不返回文件内容。它的 token 消耗极低（每个路径约 10-20 tokens），但它帮助 agent 建立了代码库的「地图」。

工具 2：Grep — 内容搜索

Grep 工具搜索文件内容，支持正则表达式，返回匹配的行及其上下文。

核心参数：

参数	说明	示例
`pattern`	正则表达式	`function\s+validate`，`TODO|FIXME`
`path`	搜索目录	`/workspace/src`
`glob`	文件过滤	`*.ts`，`*.{js,jsx}`
`output_mode`	输出格式	`content`，`files_with_matches`，`count`
`-C` / `context`	上下文行数	`3`（前后各 3 行）

实际调用示例：

Agent 发现了 `validation.ts` 和 `LoginForm.tsx`，现在想知道验证逻辑具体在哪里：

```
{
  "tool": "Grep",
  "parameters": {
    "pattern": "validateEmail|validatePassword|formError",
    "path": "src/",
    "glob": "*.{ts,tsx}",
    "output_mode": "content",
    "-C": 3
  }
}
```

返回结果：

```
src/utils/validation.ts
  12:   export function validateEmail(email: string): string | null {
  13:     if (!email) return "Email is required";
  14-    if (!email.includes("@")) return "Invalid email format";
  15:     return null;
  16:   }
  17:
  18:   export function validatePassword(password: string): string | null {
  19:     if (!password) return "Password is required";
  20-    if (password.length < 8) return "Password must be at least 8 characters";
  21:     return null;
  22:   }

src/components/LoginForm.tsx
  45:   const [formError, setFormError] = useState<string | null>(null);
  46:
  47:   const handleSubmit = () => {
  48-    const emailErr = validateEmail(email);
  49:     if (emailErr) {
  50:       setFormError(emailErr);
  51:       return;
  52:     }
  53-    // BUG: 缺少密⍰⍰⍰！
```

```
  54:     submitLogin(email, password);
  55:   };
```

通过 Grep 的上下文输出，agent 可以直接定位 bug——`handleSubmit` 函数中调用了 `validateEmail` 但跳过了 `validatePassword`。

Grep 的高级用法：

```
// 查找所有 API endpoint 定义
{
  "pattern": "router\\.(get|post|put|delete|patch)\\(",
  "glob": "*.ts",
  "output_mode": "content"
}

// 统计每个文件中 TODO 的数量
{
  "pattern": "TODO|FIXME|HACK",
  "output_mode": "count"
}

// 仅返回包含匹配的文件名（节省 token）
{
  "pattern": "import.*from.*react-router",
  "output_mode": "files_with_matches"
}
```

工具 3：Read — 完整文件阅读

Read 工具读取文件的完整内容（或指定行范围），返回带行号的文本。

核心参数：

参数	说明	示例
`file_path`	文件绝对路径	`/workspace/src/api/login.ts`
`offset`	起始行号（可选）	`40`
`limit`	读取行数（可选）	`60`

实际调用示例：

Agent 通过 Grep 锁定了 `LoginForm.tsx` 中的 bug 位置，现在要阅读完整上下文来理解修复方案：

```
{
  "tool": "Read",
  "parameters": {
```

```
    "file_path": "/workspace/src/components/LoginForm.tsx"
  }
}
```

返回整个文件内容（带行号），Agent 可以看到完整的组件结构、所有 import、state 定义、事件处理函数，从而做出准确的修复。

分段读取大文件：

对于大文件（超过 2000 行），Read 工具支持 offset 和 limit 参数进行分段读取：

```
// 只读第 100-200 行
{
  "tool": "Read",
  "parameters": {
    "file_path": "/workspace/src/generated/schema.ts",
    "offset": 100,
    "limit": 100
  }
}
```

暴力检索的工作流

一个典型的 Claude Code 风格 agent 在处理任务时，Grounding 阶段的工作流如下：

```
任□□入: "修复 LoginForm 的表□□□□bug"
     │
     ▼
┌──────────────────────────────────┐
│ Step 1: Glob 定位候□文件         │
│ pattern: **/*login*, **/*valid* │
│ Token 消耗: ~200 tokens          │
└──────────────────────────────────┘
              │
              │ 得到 7 个候□文件
              ▼
┌──────────────────────────────────┐
│ Step 2: Grep 搜索关□代□          │
│ pattern: validate|formError      │
│ □上下文 (-C 3)                   │
│ Token 消耗: ~800 tokens          │
└──────────────────────────────────┘
              │
              │ □定 2 个关□文件
              ▼
┌──────────────────────────────────┐
│ Step 3: Read □□完整文件          │
│ LoginForm.tsx (150 行)           │
│ validation.ts (80 行)            │
```

```
│ Token 消耗: ~3,000 tokens       │
└─────────────────────────────────┘
              │
              │ 完整理解代⍰⍰⍰
              ▼
    Grounding 完成，开始推理和修复
⍰⍰⍰⍰⍰⍰Token 消耗: ~4,000 tokens
```

优势与劣势

优势：

方面	说明
零配置	不需要预先构建索引、安装 LSP 服务、配置 embedding 模型
通用性	适用于任何语言、任何项目结构
可解释性	每一步搜索和阅读都是透明可审计的
鲁棒性	不依赖外部服务，不会因为索引损坏而失败
灵活性	Agent 可以根据需要动态调整搜索策略

劣势：

方面	说明
Token 开销高	读取完整文件会消耗大量 token，尤其是大文件
语义盲区	Grep 是基于文本匹配的，无法理解代码的语义结构
多跳推理困难	要追踪 `A → B → C` 的调用链，需要多轮搜索
信息遗漏	如果 agent 的搜索关键词不准确，可能遗漏关键信息
大型代码库性能	在百万行级代码库中，Glob/Grep 可能返回过多结果

Token 消耗量级参考（单次完整任务）：

```
小型⍰目 (<10K 行):   5,000 - 15,000 tokens
中型⍰目 (10K-100K):  15,000 - 50,000 tokens
大型⍰目 (>100K 行):  50,000 - 150,000 tokens
```

暴力检索的工程优化

虽然暴力检索概念简单，但可以通过工程手段大幅提升其效率：

1. 搜索结果排序

Glob 返回的文件按修改时间排序（最近修改的在前），这利用了一个启发式：最近被修改的文件与当前任务更相关。

2. 渐进式深入（Progressive Deepening）

优秀的 agent 会先用低成本操作（Glob、Grep with `files_with_matches`）建立全局认知，然后才用高成本操作（Read）获取详细内容。这类似于搜索算法中的 iterative deepening：

```
Level 0: Glob (?文件名)       → ~100 tokens
Level 1: Grep (匹配行)         → ~500 tokens
Level 2: Grep (?上下文)       → ~1,500 tokens
Level 3: Read (完整文件)        → ~3,000+ tokens
```

3. 搜索词多样化

好的 agent 不会只搜索任务描述中的关键词，而是生成多个相关的搜索词：

```
任?: "修复支付超???"
搜索?序列:
  1. payment, timeout           (直接关??)
  2. PaymentService, checkout   (可能的类名/模?名)
  3. retry, deadline, context   (相关技?概念)
  4. stripe, paypal             (可能的第三方服?名)
```

1.3 策略 2：结构化感知（Structured Perception）

超越文本匹配

暴力检索把代码当作「文本」来搜索——这丢失了代码最重要的属性：**它是有结构的**。

考虑这个场景：你搜索 `handleSubmit` 这个函数名，Grep 可能返回 20 个匹配——包括定义、调用、注释、测试、文档中的引用。但如果你只想找「这个函数的定义在哪里」或者「谁调用了这个函数」，你需要的是**结构化信息**。

结构化感知策略利用两种编译器级别的工具来理解代码：

1. **AST（Abstract Syntax Tree，抽象语法树）**
2. **LSP（Language Server Protocol，语言服务协议）**

AST：代码的骨架

AST 是源代码的树状表示，保留了程序的语法结构但去除了格式细节。

从代码到 AST：

```python
# 源代
class UserService:
    def __init__(self, db: Database):
        self.db = db

    def get_user(self, user_id: int) -> User:
        return self.db.query(User, user_id)
```

```
# 的 AST（化表示）
ClassDef: UserService
├── FunctionDef: __init__
│   ├── args: [self, db: Database]
│   └── body:
│       └── Assign: self.db = db
└── FunctionDef: get_user
    ├── args: [self, user_id: int]
    ├── returns: User
    └── body:
        └── Return: self.db.query(User, user_id)
```

AST 能提供什么信息：

信息类型	说明	示例
函数签名	函数名、参数、返回类型	`get_user(user_id: int) -> User`
类层次结构	继承关系、成员变量、方法列表	`class Admin(User): ...`
Import 依赖	当前文件依赖哪些模块	`from services.auth import verify_token`
控制流结构	条件分支、循环、异常处理	`try/except`，`if/else` 路径

信息类型	说明	示例
作用域信息	变量的定义和使用范围	局部变量 vs 类属性 vs 全局变量

用 AST 构建文件摘要：

一个关键的 grounding 优化是：不读取完整文件，而是用 AST 生成文件摘要。

```
# 用 tree-sitter 解析并生成摘要
import tree_sitter_python as tspython
from tree_sitter import Language, Parser

PY_LANGUAGE = Language(tspython.language())
parser = Parser(PY_LANGUAGE)

def generate_file_summary(source_code: bytes) -> str:
    tree = parser.parse(source_code)
    root = tree.root_node
    summary_lines = []

    for node in root.children:
        if node.type == "class_definition":
            class_name = node.child_by_field_name("name").text.decode()
            summary_lines.append(f"class {class_name}:")
            # 提取方法名（不包括方法体）
            for child in node.children:
                if child.type == "block":
                    for stmt in child.children:
                        if stmt.type == "function_definition":
                            func_name =
        stmt.child_by_field_name("name").text.decode()
                            params =
        stmt.child_by_field_name("parameters").text.decode()
                            ret = stmt.child_by_field_name("return_type")
                            ret_str = f" -> {ret.text.decode()}" if ret else ""
                            summary_lines.append(f"    def {func_name}{params}
        {ret_str}")

        elif node.type == "function_definition":
            func_name = node.child_by_field_name("name").text.decode()
            params = node.child_by_field_name("parameters").text.decode()
            summary_lines.append(f"def {func_name}{params}")

    return "\n".join(summary_lines)
```

摘要示例——一个 500 行的文件可以压缩到 30 行：

```
# 原始文件: services/payment.py (500 行)
# AST 摘要:

class PaymentService:
```

```
    def __init__(self, stripe_client, db, logger)
    def create_payment(self, user_id: int, amount: Decimal, currency: str) ->
        Payment
    def process_refund(self, payment_id: str, reason: str) -> Refund
    def get_payment_history(self, user_id: int, limit: int) -> List[Payment]
    def _validate_amount(self, amount: Decimal) -> bool
    def _notify_payment_complete(self, payment: Payment) -> None

class WebhookHandler:
    def __init__(self, payment_service, event_bus)
    def handle_stripe_webhook(self, payload: dict, signature: str) -> Response
    def _verify_signature(self, payload: dict, signature: str) -> bool
    def _process_event(self, event: StripeEvent) -> None

def create_checkout_session(user: User, items: List[CartItem]) -> str
def calculate_tax(amount: Decimal, region: str) -> Decimal
```

Token 效率对比：

```
完整文件:  ~5,000 tokens (500 行)
AST 摘要:    ~300 tokens (30 行)
⍰⍰比:    ~16:1
```

通过 AST 摘要，agent 可以用极低的 token 成本了解一个文件「有什么」，然后再决定是否需要 Read 工具阅读完整内容。

LSP：编译器级别的语义导航

LSP（Language Server Protocol）最初由 Microsoft 为 VS Code 设计，现已成为 IDE 语义功能的事实标准。LSP server 为每种语言提供统一的语义分析接口。

LSP 提供的核心能力：

LSP 方法	功能	Agent 用途
`textDocument/definition`	跳转到定义	追踪函数/类的实现位置
`textDocument/references`	查找所有引用	了解一个函数被谁调用
`textDocument/hover`	悬停信息	获取类型信息、文档注释
`textDocument/documentSymbol`	文件符号列表	生成文件结构概览
`workspace/symbol`	工作区符号搜索	按名称查找定义
`textDocument/diagnostic`	诊断信息	获取编译错误、lint 警告
`textDocument/completion`	自动补全	了解某个位置可用的 API

LSP 如何超越 Grep：

假设你搜索 `getUser`——Grep 返回所有文本匹配，但 LSP 的 `references` 方法只返回语义上的引用：

```
Grep "getUser" 的⍰果 (12 matches):
  ✓ src/services/user.ts:15    — 函数定⍰
  ✓ src/api/routes.ts:42       — ⍰用
  ✓ src/api/routes.ts:67       — ⍰用
  ✗ src/services/user.ts:3     — 注⍰中提到: "// getUser returns..."
  ✗ src/tests/user.test.ts:10  — ⍰⍰描述: "describe('getUser'..."
  ✗ docs/api.md:25             — 文档中提到
  ✗ CHANGELOG.md:12            — "Fixed getUser error handling"
  ...

LSP references 的⍰果 (3 matches):
  ✓ src/services/user.ts:15    — 定⍰
  ✓ src/api/routes.ts:42       — ⍰用
  ✓ src/api/routes.ts:67       — ⍰用
```

LSP 精确地过滤掉了注释、字符串、文档中的「假阳性」匹配——更少的 token 消耗，更高的信息密度。

OpenCode 的实现：LSP 作为 Agent 工具

OpenCode（一个开源的终端 AI 编程助手）将 LSP 集成为 agent 的工具，提供了结构化感知的工程实现范例。

OpenCode 的 LSP 工具设计：

```go
// OpenCode 中 LSP 相关的工具定⍰（⍰化）
type LSPTools struct {
    // ⍰取符号定⍰
    Definition(file string, line int, col int) -> Location

    // ⍰取所有引用
    References(file string, line int, col int) -> []Location

    // ⍰取文件的符号大⍰
    DocumentSymbols(file string) -> []Symbol

    // ⍰取⍰断信息（⍰⍰⍰⍰、警告）
    Diagnostics(file string) -> []Diagnostic

    // ⍰取⍰停信息（类型、文档）
    Hover(file string, line int, col int) -> HoverInfo
}
```

Agent 使用 LSP 的典型工作流：

```
任□: "重□□UserService.getUser 方法"
     │
     ▼
┌──────────────────────────────┐
│ Step 1: DocumentSymbols      │
│ □取 UserService 的所有方法列表  │
│ Token: ~150 tokens           │
└──────────────────────────────┘
           │
           ▼
┌──────────────────────────────┐
│ Step 2: References           │
│ □找 getUser 的所有□用方       │
│ 返回: 3 个文件, 5 个□用点      │
│ Token: ~200 tokens           │
└──────────────────────────────┘
           │
           ▼
┌──────────────────────────────┐
│ Step 3: Hover (□每个□用点)    │
│ □取类型信息, 确□参数/返回□    │
│ Token: ~300 tokens           │
└──────────────────────────────┘
           │
           ▼
┌──────────────────────────────┐
│ Step 4: Read (□定□和关□□用)  │
│ 只□取必要的代□片段            │
│ Token: ~1,000 tokens         │
└──────────────────────────────┘
           │
           ▼
    Grounding 完成
□□□□□□Token 消耗: ~1,650 tokens
    (□比暴力□索: ~4,000+ tokens)
```

结构化感知的优势与劣势

优势：

方面	说明
精确性	语义级别的定位，无假阳性
Token 效率	只获取必要信息，消耗远低于暴力检索
多跳导航	自然支持调用链追踪（A 调 B 调 C）
类型感知	理解继承、接口实现、泛型等类型系统
编译级诊断	直接获取编译错误，无需运行代码

劣势：

方面	说明
语言依赖	每种语言需要对应的 LSP server
启动开销	LSP server 启动和初始索引可能需要几秒到几十秒
可靠性问题	LSP server 可能崩溃、内存泄漏、返回不完整结果
动态语言短板	Python/JavaScript 的 LSP 类型推断不如 TypeScript/Java 精确
部署复杂度	需要在 agent 运行环境中安装和维护 LSP server
不适用于所有文件	配置文件、文档、脚本等非代码文件无法使用

各语言 LSP server 成熟度参考：

```
TypeScript (tsserver):       ★★★★★  极其成熟，微?官方??
Rust (rust-analyzer):        ★★★★★  社区?杆，性能?秀
Go (gopls):                  ★★★★☆  Google 官方??，?定
Java (Eclipse JDT LS):       ★★★★☆  功能完整，???慢
Python (Pylance/Pyright):    ★★★★☆  类型推断能力强（需类型注解）
C/C++ (clangd):              ★★★★☆  基于 LLVM，需要 compile_commands.json
PHP (Intelephense):          ★★★☆☆  功能可用
Ruby (Solargraph):           ★★★☆☆  社区??，???言限制
```

1.4 策略 3：向量检索（RAG）

为什么需要语义搜索？

暴力检索和结构化感知都有一个共同的局限：它们依赖**精确匹配**——要么是文本模式匹配（Grep），要么是符号名匹配（LSP）。但在很多场景下，agent 需要的是**语义层面的搜索**。

考虑这个任务：

> “我的应用在处理大量并发请求时响应变慢”

相关代码可能包含： - 数据库连接池配置（但代码中不包含「并发」或「慢」这些词） - 缓存层实现（关键词是 `cache`，`redis`，`ttl`） - 线程池大小配置（关键词是 `thread_pool`，`max_workers`） - 数据库查询中的 N+1 问题（没有明显关键词）

Grep 搜索 “concurrent” 或 “slow” 不会找到这些代码。但如果用语义搜索——将自然语言查询和代码片段都映射到同一个向量空间——就可以找到语义相关的代码，即使它们没有共同的关键词。

代码 Embedding 的工作原理

核心思想： 将代码片段和自然语言查询都映射到高维向量空间中，使得语义相似的内容在向量空间中距离更近。

```
            向量空⍰（768⍰/1536⍰）

"⍰理并⍰⍰求"  ──→  [0.23, -0.14, 0.87, ...]  ┐
                                              │  余弦相似度: 0.82
"class ConnectionPool:                        │
   max_connections = 50"  ──→  [0.19, -0.11, 0.91, ...] ┘
```

主流代码 Embedding 模型：

模型	维度	特点	适用场景
OpenAI `text-embedding-3-small`	1536	通用文本 + 代码，API 调用	SaaS 方案，快速原型
OpenAI `text-embedding-3-large`	3072	更高精度，更高成本	对精度要求高的场景
Voyage `voyage-code-3`	1024	专门为代码优化	代码搜索专用场景
`BAAI/bge-code-v1`	1024	开源，支持本地部署	隐私敏感场景
StarCoder Embeddings	768	开源，代码预训练	预算有限的本地部署

代码分块策略（Chunking）

Embedding 模型通常有输入长度限制（512-8192 tokens）。如何将代码库切分成合适大小的「块」（chunk），直接影响检索质量。

策略 1：按函数/类分块

```
# 将每个函数/类作⍰一个独立的 chunk
def chunk_by_function(source: str, language: str) -> List[Chunk]:
    tree = parser.parse(source.encode())
    chunks = []

    for node in tree.root_node.children:
        if node.type in ["function_definition", "class_definition"]:
            chunk_text = source[node.start_byte:node.end_byte]
            chunks.append(Chunk(
                text=chunk_text,
                metadata={
                    "type": node.type,
                    "name": node.child_by_field_name("name").text.decode(),
                    "start_line": node.start_point[0],
                    "end_line": node.end_point[0],
                }
            ))

    return chunks
```

优势：语义完整，边界自然。劣势：大函数可能超过 token 限制。

策略 2：按文件分块（带摘要）

```
# 整个文件作⍰一个 chunk，⍰大文件使用 AST 摘要
def chunk_by_file(file_path: str, max_tokens: int = 2048) -> Chunk:
    source = read_file(file_path)
    token_count = count_tokens(source)

    if token_count <= max_tokens:
        return Chunk(text=source, metadata={"file": file_path})
    else:
        # 大文件用 AST 摘要替代
        summary = generate_ast_summary(source)
        return Chunk(text=summary, metadata={
            "file": file_path,
            "is_summary": True,
            "original_lines": count_lines(source),
        })
```

策略 3：滑动窗口分块

```python
# 固定大小滑窗口，重叠
def chunk_by_sliding_window(
    source: str,
    window_size: int = 60,   # 60 行
    overlap: int = 10         # 10 行重叠
) -> List[Chunk]:
    lines = source.split("\n")
    chunks = []

    for i in range(0, len(lines), window_size - overlap):
        chunk_lines = lines[i:i + window_size]
        chunks.append(Chunk(
            text="\n".join(chunk_lines),
            metadata={
                "start_line": i,
                "end_line": min(i + window_size, len(lines)),
            }
        ))

    return chunks
```

各策略对比：

策略	语义完整性	实现复杂度	适用场景
按函数/类分块	★★★★★	★★★☆☆（需 AST 解析）	结构良好的代码库
按文件分块	★★★★☆	★★☆☆☆	小文件为主的项目
滑动窗口	★★☆☆☆	★☆☆☆☆	快速原型，不规则文件
混合分块	★★★★★	★★★★☆	生产环境

向量数据库选型

数据库	类型	适用规模	特点
FAISS	库（in-process）	小到中型	Meta 开源，高性能，纯内存
ChromaDB	嵌入式数据库	小到中型	简单易用，Python 原生
Qdrant	独立服务	中到大型	Rust 实现，支持过滤
Weaviate	独立服务	大型	支持混合搜索（向量 + 关键词）
Pinecone	云服务	大型	全托管，无需运维

一个最小化的 RAG 实现示例：

```python
import chromadb
from openai import OpenAI

# 初始化
client = OpenAI()
chroma = chromadb.Client()
collection = chroma.create_collection("codebase")

# 索引⍰段：将代⍰⍰分⍰并存入向量数据⍰
def index_codebase(root_dir: str):
    for file_path in glob.glob(f"{root_dir}/**/*.py", recursive=True):
        source = open(file_path).read()
        chunks = chunk_by_function(source, "python")

        for i, chunk in enumerate(chunks):
            chunk_id = f"{file_path}:{i}"
            embedding = client.embeddings.create(
                model="text-embedding-3-small",
                input=chunk.text
            ).data[0].embedding

            collection.add(
                ids=[chunk_id],
                embeddings=[embedding],
                documents=[chunk.text],
                metadatas=[chunk.metadata]
            )

# ⍰索⍰段：根据⍰⍰找到最相关的代⍰片段
def retrieve(query: str, top_k: int = 5) -> List[dict]:
    query_embedding = client.embeddings.create(
        model="text-embedding-3-small",
        input=query
    ).data[0].embedding

    results = collection.query(
        query_embeddings=[query_embedding],
        n_results=top_k,
    )

    return [
        {
            "code": doc,
            "metadata": meta,
            "distance": dist,
        }
        for doc, meta, dist in zip(
            results["documents"][0],
            results["metadatas"][0],
            results["distances"][0],
        )
    ]
```

语义漂移（Semantic Drift）问题

RAG 在代码检索中面临一个独特的挑战：**语义漂移**。

代码中的语义关系与自然语言不同：

```
# ⍰两个函数⍰⍰高度相关，但文本相似度很低
def authenticate_user(username: str, password: str) -> Token:
    hashed = bcrypt.hash(password)
    user = db.users.find_one({"username": username})
    if user and bcrypt.verify(password, user["password_hash"]):
        return generate_jwt(user["id"])
    raise AuthenticationError("Invalid credentials")

def check_permission(token: str, resource: str, action: str) -> bool:
    claims = decode_jwt(token)
    role = get_user_role(claims["user_id"])
    return role.has_permission(resource, action)
```

对于查询「用户认证流程」，embedding 模型可能正确召回 `authenticate_user`，但遗漏 `check_permission`——后者是认证流程的下游，但其文本表示与「认证」的语义距离较远。

缓解策略：

1. **增强元数据：** 在 chunk 中添加文件路径、类名、调用关系作为额外上下文
2. **上下文窗口扩展：** 检索到某个函数后，自动包含其所在类的其他方法
3. **多查询策略：** 将原始查询改写为多个子查询，合并结果

```
# 多⍰⍰策略示例
def multi_query_retrieve(original_query: str, top_k: int = 5) -> List[dict]:
    # 用 LLM 生成多个角度的⍰⍰
    expanded_queries = llm.generate(
        f"将以下⍰⍰改写⍰ 3 个不同角度的搜索⍰⍰：\n{original_query}"
    )
    # 例如:
    # ["用⍰登⍰⍰⍰", "JWT token 生成和⍰⍰", "权限⍰⍰和授权"]

    all_results = []
    seen_ids = set()

    for query in expanded_queries:
        results = retrieve(query, top_k=top_k)
        for r in results:
            if r["id"] not in seen_ids:
                all_results.append(r)
                seen_ids.add(r["id"])
```

```
    # 按⍰合相似度排序，取 top_k
    all_results.sort(key=lambda x: x["distance"])
    return all_results[:top_k]
```

RAG 的优势与劣势

优势：

方面	说明
规模扩展性	百万行代码库也可以在毫秒级检索
语义理解	可以处理自然语言查询，跨越关键词限制
一次索引多次查询	索引构建后，查询成本极低
跨文件关联	自然地发现语义相关但位置分散的代码

劣势：

方面	说明
语义漂移	Embedding 可能误判代码的语义距离
索引维护	代码修改后需要更新索引
冷启动成本	首次索引大型代码库需要较长时间和 API 调用
Embedding 质量不稳定	不同语言、不同风格的代码 embedding 质量差异大
无法处理精确查询	搜索特定函数名时不如 Grep/LSP 精确

1.5 策略 4：混合方案（Hybrid Approaches）

为什么需要混合？

前三种策略各有所长：

```
              精确性    ⍰⍰性    Token效率    ⍰⍰成本    大型⍰支持
暴力⍰索(Glob/Grep)  ★★★☆☆    ★☆☆☆☆     ★★☆☆☆       ★★★★★       ★★☆☆☆
⍰⍰化感知(AST/LSP)   ★★★★★    ★★☆☆☆     ★★★★★       ★★☆☆☆       ★★★★☆
向量⍰索(RAG)        ★★☆☆☆    ★★★★★     ★★★★☆       ★★★☆☆       ★★★★★
```

没有单一策略能覆盖所有场景。实践中最有效的方案通常是**分层混合**——用不同策略处理不同阶段的信息需求。

Agentless 方法：两步式定位

Agentless（Xia et al., 2024）提出了一个优雅的两步框架，证明不需要复杂的 agent 循环，简单的定位+修复就能取得很好的效果。

Step 1: Localization（定位）

```
⍰入: Bug ⍰告 + 完整的代⍰⍰⍰⍰

⍰⍰ 1: 文件定位
  LLM ⍰入: ⍰⍰目⍰⍰ + bug 描述
  LLM ⍰出: 可能相关的文件列表 (top-5)

⍰⍰ 2: 类/函数定位
  LLM ⍰入: 相关文件的 AST 摘要 (函数⍰名列表) + bug 描述
  LLM ⍰出: 可能相关的函数/类列表 (top-3)

⍰⍰ 3: 行⍰定位
  LLM ⍰入: 相关函数的完整代⍰ + bug 描述
  LLM ⍰出: 需要修改的具体行号范⍰
```

Step 2: Repair（修复）

```
⍰入: 定位⍰果 + 周⍰上下文
LLM ⍰出: 具体的代⍰⍰丁 (patch)
```

Agentless 的关键洞察：

```
⍰⍰⍰Agent 工作流:
 搜索 → ⍰⍰ → 搜索 → ⍰⍰ → 搜索 → ... → 修复
  (多⍰交互, 每⍰消耗 tokens, 可能⍰入搜索死循⍰)

Agentless 工作流:
 目⍰⍰ → 文件定位 → 摘要 → 函数定位 → 代⍰ → 行⍰定位 → 修复
  (固定步⍰, 可⍰⍰的 token 消耗, 不会陷入循⍰)
```

实现示例（简化）：

```python
def agentless_localize(bug_report: str, repo_root: str) -> dict:
    # ⍰⍰ 1: 文件定位
    dir_tree = generate_directory_tree(repo_root, max_depth=3)
    file_prompt = f"""
  以下是⍰⍰目⍰⍰⍰：
    {dir_tree}

    Bug ⍰告：
    {bug_report}
```

```
⍰⍰⍰⍰⍰列出最可能包含 bug 的 5 个文件路径。
    """
    candidate_files = llm.generate(file_prompt)  # ["src/auth/login.py", ...]

    # ⍰⍰ 2: 函数/类定位
    summaries = {}
    for f in candidate_files:
        source = read_file(f)
        summaries[f] = generate_ast_summary(source)

    func_prompt = f"""
  以下是候⍰文件的⍰⍰摘要：
    {format_summaries(summaries)}

    Bug ⍰告：
    {bug_report}

⍰⍰⍰⍰⍰列出最可能包含 bug 的 3 个函数/类。
    """
    candidate_funcs = llm.generate(func_prompt)

    # ⍰⍰ 3: 行⍰定位
    code_snippets = extract_function_code(candidate_funcs)
    line_prompt = f"""
  以下是候⍰函数的完整代⍰：
    {code_snippets}

    Bug ⍰告：
    {bug_report}

⍰⍰⍰⍰⍰指出需要修改的具体行号范⍰。
    """
    target_lines = llm.generate(line_prompt)

    return {
        "files": candidate_files,
        "functions": candidate_funcs,
        "lines": target_lines,
    }
```

层级式混合方案

将 Agentless 的思想推广，我们可以设计一个通用的层级式 grounding 框架：

```
┌──────────────────────────────────────────────────────────┐
│                      Layer 0: 全局索引                    │
│  方法: 目⍰⍰ + 文件⍰⍰RAG 索引                              │
│  目的: 快速⍰小范⍰到 10-20 个候⍰文件                       │
│  Token 消耗: ~500 tokens                                  │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
```

```
│                    Layer 1: 文件⍰感知                        │
│  方法: AST 摘要 + DocumentSymbols (LSP)                      │
│  目的: ⍰别 3-5 个关⍰函数/类                                  │
│  Token 消耗: ~1,000 tokens                                   │
└──────────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────────┐
│                    Layer 2: 符号⍰⍰航                        │
│  方法: LSP references/definition + Grep 上下文               │
│  目的: 理解⍰用关系、类型信息                                 │
│  Token 消耗: ~1,500 tokens                                   │
└──────────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────────┐
│                    Layer 3: 精⍰⍰⍰                           │
│  方法: Read 完整函数/文件                                    │
│  目的: ⍰取修改所需的完整上下文                               │
│  Token 消耗: ~2,000-5,000 tokens                             │
└──────────────────────────────────────────────────────────────┘


⍰⍰: ~5,000-8,000 tokens (vs 暴力⍰索的 15,000-50,000)
```

RAG + Structured 混合

对于大型代码库，一个高效的组合方案是：

```python
def hybrid_grounding(query: str, repo: Repository) -> Context:
    # Phase 1: RAG 粗⍰ — O(1) ⍰⍰复⍰度
    # 在⍰建的向量索引中找到 top-20 候⍰⍰chunks
    candidate_chunks = rag_retrieve(query, top_k=20)

    # Phase 2: ⍰⍰化⍰⍰ — 用 LSP/AST ⍰⍰候⍰
    # 排除 false positive，⍰充 false negative
    verified = []
    for chunk in candidate_chunks:
        # 用 LSP ⍰⍰⍰个 chunk 是否在相关的⍰用⍰上
        refs = lsp.references(chunk.file, chunk.function_name)
        if is_relevant(refs, query):
            verified.append(chunk)
            # 追踪⍰用⍰上的相关函数（RAG 可能⍰漏的）
            for ref in refs:
                related = lsp.definition(ref.file, ref.line, ref.col)
                if related not in verified:
                    verified.append(related)

    # Phase 3: 精⍰⍰⍰ — 只⍰取⍰⍰通⍰的代⍰
    context = []
    for chunk in verified[:10]:  # 限制最⍰数量
        code = read_function(chunk.file, chunk.start_line, chunk.end_line)
        context.append(code)

    return Context(chunks=context)
```

混合方案的设计原则

在设计混合 grounding 方案时，有几个关键原则：

原则 1：漏斗模型（Funnel Model）

每一层都应该是上一层的精化——从广到窄，从粗到细：

```
全部文件 (10,000) → 候⍰文件 (20) → 候⍰函数 (5) → 关⍰代⍰ (200 行)
     100%              0.2%            0.05%           0.002%
```

原则 2：Recall 优先，Precision 后补

早期阶段应优先保证召回率（不遗漏关键信息），后期阶段再优化精确率（剔除无关信息）。因为遗漏关键信息会导致错误的修复，而多看一些无关信息只是浪费一些 token。

```
Layer 0 (RAG):     Recall > 90%, Precision ~ 30%   → 广撒网
Layer 1 (AST):     Recall > 85%, Precision ~ 60%   → 初步⍰⍰
Layer 2 (LSP):     Recall > 80%, Precision ~ 85%   → 精确⍰航
Layer 3 (Read):    Recall = 100%, Precision = 100%  → 完整上下文（在已定位范⍰内）
```

原则 3：优雅降级（Graceful Degradation）

混合系统中任何一个组件都可能失败——LSP server 崩溃、向量索引过期、AST 解析失败。方案应该设计为：任何一层失败时，可以用更暴力但更可靠的方法替代：

```python
def resilient_grounding(query: str) -> Context:
    # ⍰⍰⍰RAG 粗⍰
    try:
        candidates = rag_retrieve(query, top_k=20)
    except RAGError:
        # 降⍰: 用 Glob + Grep 替代
        candidates = glob_grep_fallback(query)

    # ⍰⍰⍰LSP 精确⍰航
    try:
        refined = lsp_refine(candidates)
    except LSPError:
        # 降⍰: 用 AST 摘要替代
        try:
            refined = ast_refine(candidates)
        except ASTError:
            # 再降⍰: 直接⍰取文件
            refined = candidates
```

```
    return build_context(refined)
```

1.6 上下文窗口管理

基本约束

上下文窗口是 coding agent 最根本的物理约束。无论 grounding 策略多么精妙，最终所有信息都必须塞进有限的上下文窗口中。

当前主流模型的上下文窗口：

模型	上下文窗口	约等于代码行数	约等于文件数（200行/文件）
Claude 3.5 Sonnet	200K tokens	~150,000 行	~750 个文件
GPT-4o	128K tokens	~96,000 行	~480 个文件
Claude 3 Opus	200K tokens	~150,000 行	~750 个文件
Gemini 1.5 Pro	1M tokens	~750,000 行	~3,750 个文件
Claude Opus 4	200K tokens	~150,000 行	~750 个文件

但实际可用空间远小于理论值：

```
Claude 200K 上下文窗口分配:

System Prompt:          ~5,000 tokens  (2.5%)
工具定?:                ~3,000 tokens  (1.5%)
???史:                  ~30,000 tokens (15%)
当前?工具?果:          ~50,000 tokens (25%)
──────────────────────────────────────────
已占用:                 ~88,000 tokens (44%)
可用于新代?上下文:      ~112,000 tokens (56%)

??上，考?到?出 token ?留:
可用于?入:              ~100,000 tokens (~50%)
```

策略 1：信息压缩（Summarization）

当读取的代码超过上下文预算时，需要对信息进行压缩。

层级式压缩：

```
□□□别 0（无□□）：完整源代□
  → 每个文件 ~2000-5000 tokens

□□□别 1（去注□）：移除注□和空行
  → 减少 20-30% tokens

□□□别 2（AST 摘要）：只保留□名和□□
  → 减少 80-90% tokens

□□□别 3（自然□言摘要）：用一段□描述文件功能
  → 减少 95%+ tokens
```

实现示例：

```python
def compress_context(files: List[FileContent], budget: int) -> List[str]:
    """根据 token □算自适□□□文件内容"""
    total_tokens = sum(count_tokens(f.content) for f in files)

    if total_tokens <= budget:
        # □算充足，不需要□□
        return [f.content for f in files]

    # 按重要性排序（直接相关的文件排前面）
    files.sort(key=lambda f: f.relevance_score, reverse=True)

    result = []
    remaining_budget = budget

    for f in files:
        file_tokens = count_tokens(f.content)

        if file_tokens <= remaining_budget:
            # □算足够，保留完整内容
            result.append(f.content)
            remaining_budget -= file_tokens
        elif remaining_budget > 200:
            # □算不够完整内容，用 AST 摘要
            summary = generate_ast_summary(f.content)
            summary_tokens = count_tokens(summary)
            if summary_tokens <= remaining_budget:
                result.append(f"# [AST Summary] {f.path}\n{summary}")
                remaining_budget -= summary_tokens
        else:
            # □算耗尽，跳□剩余文件
            break
```

```
    return result
```

策略 2：选择性包含（Selective Inclusion）

不是所有信息都同等重要。设计一个优先级系统来决定哪些信息必须包含，哪些可以省略。

信息优先级矩阵：

优先级	信息类型	示例
P0 (必须)	直接修改目标	要修复的函数本身
P0 (必须)	错误信息	Stack trace, 编译错误
P1 (重要)	直接依赖	被修改函数调用的其他函数
P1 (重要)	类型定义	参数和返回值的类型/接口定义
P2 (有用)	调用方	调用被修改函数的上游代码
P2 (有用)	测试用例	相关的测试代码
P3 (补充)	配置文件	相关的配置
P3 (补充)	文档	API 文档、注释
P4 (可选)	类似代码	其他模块的类似实现（作参考）

策略 3：滑动窗口与对话管理

在多轮对话中，历史信息会持续消耗上下文。Claude Code 采用的策略是**自动压缩历史**：

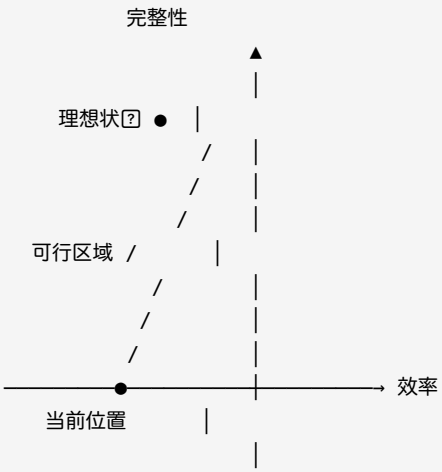
```
□□流程：
  Turn 1: 用□描述任□ + Agent 搜索文件
  Turn 2: Agent □□文件 + 提出修复方案
  Turn 3: 用□确□ + Agent □行修复
  Turn 4: Agent □行□□ + □告□果
  ...

当上下文接近限制□，自□□□早期 turn:
  Turn 1-3: [□□□摘要] "已定位并修复 LoginForm.tsx 中的□□□bug"
  Turn 4: [保留完整内容] □□□出...
  Turn 5: [保留完整内容] 当前操作...
```

Claude Code 的具体实现行为：

```
上下文到□□□□:
1. 系□自□□□□早的□□□史
2. 保留最近的完整交互
3. 关□信息（如文件修改□□）在□□中被保留
4. Agent 可以通□重新□取文件来恢复□失的上下文
```

完整性 vs 效率的张力

这是 grounding 设计中最核心的 tension：



完整性： 确保上下文中包含所有解决任务需要的信息。遗漏任何关键信息都可能导致错误的输出。

效率： 最小化 token 消耗，降低成本和延迟。

这两个目标天然矛盾。追求完整性意味着读取更多代码（更多 token），追求效率意味着激进裁剪（可能遗漏）。

工程中的务实选择：

```python
# 在不同□景下□整策略
def choose_grounding_strategy(task: Task, codebase: Codebase) -> Strategy:
    if task.is_simple_fix and codebase.size < 10_000:
        # 小型□目的□□修复 → 暴力□索足够
        return BruteForceStrategy(max_files=10)

    elif task.is_refactoring and codebase.has_lsp:
        # 有 LSP 的重□任□ → □□化感知最合适
        return StructuredStrategy(use_lsp=True)

    elif codebase.size > 100_000:
        # 大型代□□ → 必□用 RAG □小范□
        return HybridStrategy(
            coarse=RAGRetriever(top_k=20),
```

```
            fine=BruteForceStrategy(max_files=5),
        )

    else:
        # 默?: 暴力?索 + AST 摘要
        return BruteForceStrategy(
            use_ast_summary=True,
            max_files=15,
        )
```

关键论文导读

1. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering

论文信息： Yang et al., Princeton NLP Group, 2024

核心贡献：

这篇论文的标题就揭示了其核心论点：**Agent-Computer Interface (ACI)** 是 coding agent 性能的关键决定因素。

ACI 是类比人机交互（HCI）提出的概念——如果 HCI 关注的是人如何与计算机交互，ACI 关注的就是 AI agent 如何与计算机交互。

关键实验发现：

1. **文件查看方式的影响：** 将文件查看从”滚动翻页”改为”搜索跳转”，在 SWE-bench 上的解决率提高了超过 10 个百分点。原因是滚动翻页容易让模型迷失在大文件中，而搜索跳转允许模型直接定位关键代码。

2. **编辑工具的影响：** 提供结构化的编辑命令（指定行号范围 + 替换内容）比让模型生成完整的 diff 补丁更可靠——减少了格式错误和偏移量计算错误。

3. **错误反馈的影响：** 当命令执行失败时，提供清晰的错误信息和修复建议，比简单返回原始 traceback 显著提升了 agent 的恢复能力。

对 Grounding 的启示：

```
ACI ??的三个?度：
```

```
1. 信息□取方式（Perception）  ← 本模□的主□
   - 提供什么搜索工具
   - 搜索□果以什么格式呈□
   - 是否允□□□式探索

2. 操作方式（Action）
   - □□工具的接口□□
   - 命令的粒度和原子性

3. 反□方式（Feedback）
   - □行□果的呈□格式
   - □□信息的□□化程度
```

推荐阅读重点： 论文的 Section 3（ACI Design）和 Section 4.2（Ablation Studies）。

2. Agentless: Demystifying LLM-based Software Engineering Agents

论文信息： Xia et al., UIUC, 2024

核心贡献：

Agentless 的贡献在于：**证明了不需要复杂的 agent 架构，仅靠固定的两步流程（定位 + 修复）就可以取得与复杂 agent 相当甚至更好的性能。**

关键设计选择：

1. **层级定位（Hierarchical Localization）：**
   - 不使用搜索工具（不需要 Grep/Glob）
   - 直接给 LLM 提供仓库结构，让它推理出可能的文件位置
   - 每一层用 AST 摘要（函数签名）代替完整代码
2. **采样与投票（Sampling & Voting）：**
   - 对每个 bug 生成多个候选补丁
   - 用回归测试过滤无效补丁
   - 用多数投票选择最终补丁

关键数据：

```
SWE-bench Lite 性能□比（2024 年□文□表□）：

  SWE-agent (GPT-4):     18.0%
  Agentless (GPT-4):     27.3%  (+9.3%)

 成本□比:
```

```
  SWE-agent:    ~$3.50/□□ (多□□agent 循□)
  Agentless:    ~$0.34/□□ (固定 2-step pipeline)

成本效率:  Agentless 的每美元解决率 > SWE-agent 的 10x
```

对 Grounding 的启示：

Agentless 证明了两个反直觉的结论：

1. **目录树 + AST 摘要可以替代搜索工具。** LLM 在看到仓库结构后，有很强的文件定位能力——这意味着「看目录树」本身就是一种高效的 grounding 策略。

2. **固定的 grounding pipeline 可以胜过动态搜索。** Agent 的动态搜索看似灵活，但容易陷入循环或遗漏关键文件。固定的层级式 pipeline 虽然不灵活，但可靠性更高。

推荐阅读重点： 论文的 Section 3（Approach）和 Table 1（与其他方法的对比）。

实操环节

练习 1：比较暴力检索 vs 结构化感知

目标： 在同一个 bug 修复任务上，分别用 Glob+Grep+Read（Claude Code 风格）和 LSP（OpenCode 风格）进行 grounding，对比二者的效果。

任务描述：

在一个中等规模的 TypeScript 项目（约 20,000 行代码）中，修复以下 bug：

> 用户报告：在编辑个人资料页面，修改邮箱地址后点击保存，页面显示保存成功但实际邮箱未更新。

步骤 A：暴力检索 Grounding

1. 使用 Glob 搜索相关文件：

```
模式 1: **/*profile*
模式 2: **/*email*
模式 3: **/*user*update*
```

2. 使用 Grep 搜索关键代码：

```
模式 1: updateEmail|changeEmail|saveProfile
模式 2: email.*update|update.*email
模式 3: profileForm|handleSave
```

3. 使用 Read 阅读关键文件（记录读取的文件数和总行数）。

记录表格：

指标	暴力检索
Glob 调用次数	
Grep 调用次数	
Read 调用次数	
搜索结果总数	
阅读代码总行数	
估算 Token 消耗	
是否找到 bug 根因	
找到 bug 的步骤数	

步骤 B：结构化感知 Grounding

1. 使用 LSP `workspace/symbol` 搜索 `updateProfile`，`updateEmail` 等符号
2. 使用 LSP `references` 追踪调用链
3. 使用 LSP `definition` 跳转到实现
4. 使用 LSP `hover` 检查类型信息
5. 最后用 Read 阅读关键代码

记录表格：

指标	结构化感知
LSP 调用次数	
Read 调用次数	
搜索结果总数（精确匹配）	

指标	结构化感知
阅读代码总行数	
估算 Token 消耗	
是否找到 bug 根因	
找到 bug 的步骤数	

步骤 C：对比分析

完成两种方法后，填写对比表：

维度	暴力检索	结构化感知	分析
Token 消耗			哪个更省 token？差多少？
信息精确度			哪个返回的无关信息更少？
覆盖完整性			哪个遗漏了关键信息？
操作步骤数			哪个更快到达目标？
认知负担			哪个更容易理解结果？

练习 2：测量不同策略的 Token 消耗和信息覆盖率

目标： 量化不同 grounding 策略在大型代码库上的 Token-信息 trade-off。

准备工作：

选择一个开源项目（推荐 fastapi、express 或类似中大型项目），准备以下 5 个不同类型的任务：

```
任□ 1（□□定位）："找到 XXX 函数的定□位置"
任□ 2（□用□追踪）："找到所有□用 XXX 的地方"
任□ 3（跨文件理解）："理解 XXX 功能的完整□□"
任□ 4（Bug 定位）："XXX 功能在 YYY 条件下出□"
任□ 5（重□范□□估）："如果修改 XXX 的接口，需要改哪些地方"
```

对每个任务，分别尝试以下策略：

策略 A: 纯 Grep + Read

```
□□：搜索次数，□取文件数，□取行数，□□token 估算
```

策略 B: AST 摘要 + 定向 Read

```
□□: 摘要文件数, □取文件数, □取行数, □□token 估算
```

策略 C: 目录树 + LLM 定位 + Read（Agentless 式）

```
□□: LLM □用次数, □取文件数, □取行数, □□token 估算
```

汇总表格:

任务	策略 A Token	策略 B Token	策略 C Token	最优策略	信息完整度评分 (1-5)
任务1					
任务2					
任务3					
任务4					
任务5					

分析问题: 1. 哪种类型的任务最受益于结构化感知? 2. 在什么情况下暴力检索反而更高效? 3. 如何根据任务类型动态选择策略?

本模块小结

核心要点回顾

```
┌──────────────────────────────────────────────────────────┐
│                    Grounding 策略空□                      │
```

```
├──────────────┼──────────────────────────────────────────┤
│  暴力⍰索      │ Glob/Grep/Read — ⍰⍰通用，token 开⍰高    │
│  ⍰⍰化感知    │ AST/LSP — 精确高效，需要⍰言支持          │
│  向量⍰索      │ RAG — ⍰⍰搜索，适合大型⍰，有漂移⍰⍰       │
│  混合方案      │ 分⍰⍰合 — ⍰践最⍰，⍰⍰复⍰度高            │
├──────────────┼──────────────────────────────────────────┤
│  核心 trade-off: 信息完整性 vs Token 效率                 │
│  核心⍰束: 上下文窗口大小                                  │
│  核心原⍰: ACI ⍰⍰ > 模型⍰⍰ (SWE-agent)                   │
└──────────────────────────────────────────────────────────┘
```

关键概念总结

概念	定义	重要性
ACI (Agent-Computer Interface)	Agent 与开发环境交互的接口设计	决定 agent 上限的关键因素
Grounding	将 agent 锚定在真实代码信息上的过程	推理质量的前提
Token Budget	上下文窗口中可用于代码的 token 量	所有策略的硬约束
AST 摘要	用语法树生成代码结构摘要	压缩比可达 16:1
语义漂移	向量搜索中语义距离判断不准确的现象	RAG 方案的主要风险
层级定位	从粗到细逐步缩小目标范围	混合方案的核心模式
优雅降级	高级策略失败时回退到基础策略	生产系统的必要设计

思考题

思考题 1：设计选择

假设你要为一个 50 万行的 Java 微服务项目（200+ 个服务）构建一个 coding agent。你会选择什么 grounding 策略组合？为什么？请考虑以下因素： - 项目有完整的 Maven 构建配置 - Java LSP (Eclipse JDT LS) 可用但启动慢（约 30 秒） - 大部分任务是 bug 修复和功能添加 - 需要支持跨服务的修改

思考题 2：Token 优化

当上下文窗口中 60% 的 token 被「可能相关但不一定需要」的代码占据时，你会如何优化？提出至少三种策略，并分析各自的风险。

思考题 3：Grounding 的未来

随着模型上下文窗口持续增长（Gemini 已支持 1M+ tokens），暴力检索是否会成为唯一需要的策略？结构化感知和 RAG 是否会变得不必要？请论证你的观点。

思考题 4：评估指标

如何量化评估一个 grounding 策略的好坏？提出一个包含至少 4 个维度的评估框架，并为每个维度设计一个可计算的指标。

思考题 5：从 SWE-agent 到实践

SWE-agent 论文发现 ACI 设计比模型选择更重要。但这个结论是否普遍成立？在什么条件下，模型能力会重新成为瓶颈？提出你的假设并设计一个验证实验。